

Full Stack Development with MERN

Cab Booking System

1. Introduction

Project Title: CarGo - Cab Booking System

Team Members: Mutyala Mohini Sri Mani

Roles- Frontend Development, BackDevelopment, DataBase Integration, Testing and Documentation.

2. Project Overview

Purpose

The purpose of CarGo is to provide a simple, reliable, and user-friendly online cab booking platform that enables users to book rides conveniently while helping administrators manage cab operations efficiently through a centralized system.

The application allows users to register, log in, search for available cabs, book rides, track booking status, and view booking history. Administrators can manage cabs, drivers, users, and bookings through an administrative dashboard.

Developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js), CarGo aims to improve accessibility, reduce manual booking processes, and provide a seamless ride-booking experience for both customers and service providers.

Features

User Features

- User Registration and Login
- Browse Available Cabs
- Cab Booking
- Fare Estimation
- Booking History
- Booking Status Tracking

Driver Features

- Driver Login
- View Assigned Trips

- Update Availability Status
- Trip History

Admin Features

- Admin Login
- Manage Cabs
- Manage Drivers
- Manage Users
- Manage Bookings
- Dashboard Analytics

3. Architecture

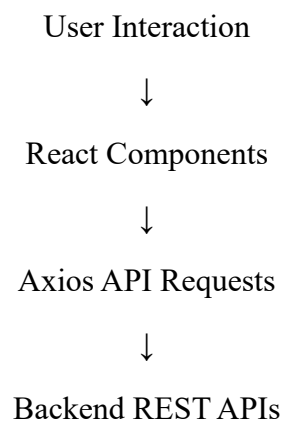
3.1 Frontend Architecture

The frontend of CarGo is developed using **React.js** and follows a **component-based architecture** to improve reusability, maintainability, and scalability.

The frontend consists of:

- **Pages** for different application screens such as Home, Login, Register, Booking, and Dashboard.
- **Reusable Components** such as Navbar, CabCard, DriverCard, Loader, and Sidebar.
- **Context API** for global state management.
- **Axios** for communication with backend APIs.
- **React Router DOM** for client-side routing and navigation.

Frontend Flow:



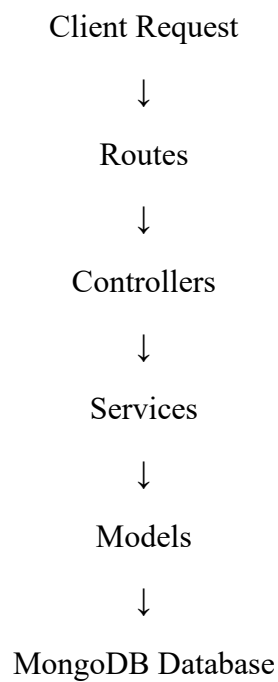
3.2 Backend Architecture

The backend of CarGo is developed using **Node.js** and **Express.js** and follows a layered architecture.

The backend is responsible for:

- Processing user requests.
- Implementing business logic.
- Managing authentication and authorization.
- Performing database operations.
- Returning responses to the frontend.

Backend Flow



Backend Components

- Routes
- Controllers
- Services
- Middleware
- Models

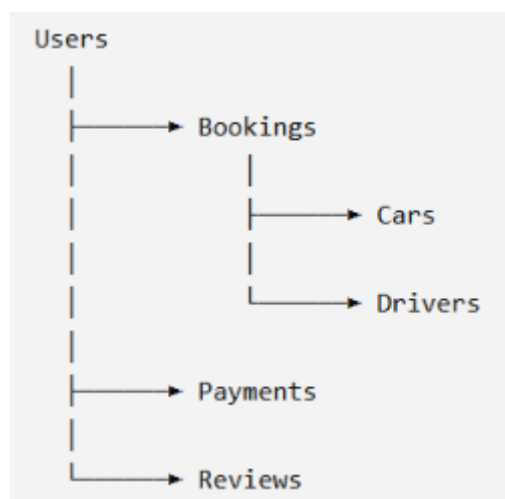
3.3 Database Architecture

The application uses **MongoDB Atlas** as its cloud database service.

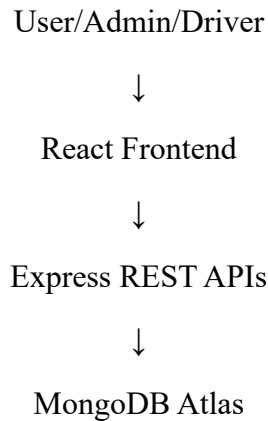
The following collections are maintained:

Collection	Purpose
Users	Stores customer details
Admins	Stores administrator details
Drivers	Stores driver information
Cars	Stores vehicle information
Bookings	Stores booking records
Payments	Stores payment details
Notifications	Stores notifications
Reviews	Stores customer feedback

Database Relationships



System Architecture Diagram



4. Setup Instructions

4.1 Prerequisites

Before running the project, ensure that the following software and tools are installed on the system:

- Node.js (v18 or above recommended)
- npm (Node Package Manager)
- MongoDB Atlas Account
- Git
- Visual Studio Code
- Postman (for API testing)

4.2 Clone the Repository

Clone the project repository using Git:

```
git clone < https://github.com/mohinimutyala/cab-booking-mern>
```

Move into the project directory:

```
cd cab-booking
```

4.3 Frontend Setup

Navigate to the client directory:

```
cd client
```

Install frontend dependencies:

```
npm install
```

Start the frontend development server:

```
npm run dev
```

The frontend application will run on:

<http://localhost:5173>

4.4 Backend Setup

Open another terminal and navigate to the server directory:

```
cd server
```

Install backend dependencies:

```
npm install
```

Start the backend server:

```
npm run dev
```

The backend server will run on:

<http://localhost:8000>

```
node server.js
```

4.5 Environment Variables

Create a .env file inside the server directory and add the following variables:

```
PORT=8000
```

```
MONGO_URI=your_mongodb_connection_string
```

```
JWT_SECRET=your_jwt_secret_key
```

Example:

```
PORT=8000
```

```
MONGO_URI=mongodb+srv://username:password@cluster.mongodb.net/ucab
```

```
JWT_SECRET=mysecretkey123
```

4.6 Verify Installation

After starting both servers:

- Frontend should open successfully on <http://localhost:5173>

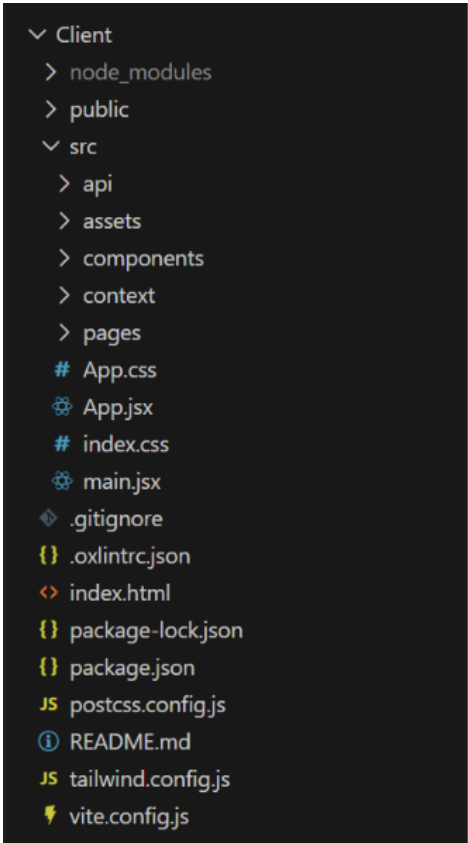
- Backend should display:

Serverrunning on port 8000
MongoDB Connected Successfully

If both messages appear, the application setup is complete and ready for use.

5. Folder Structure

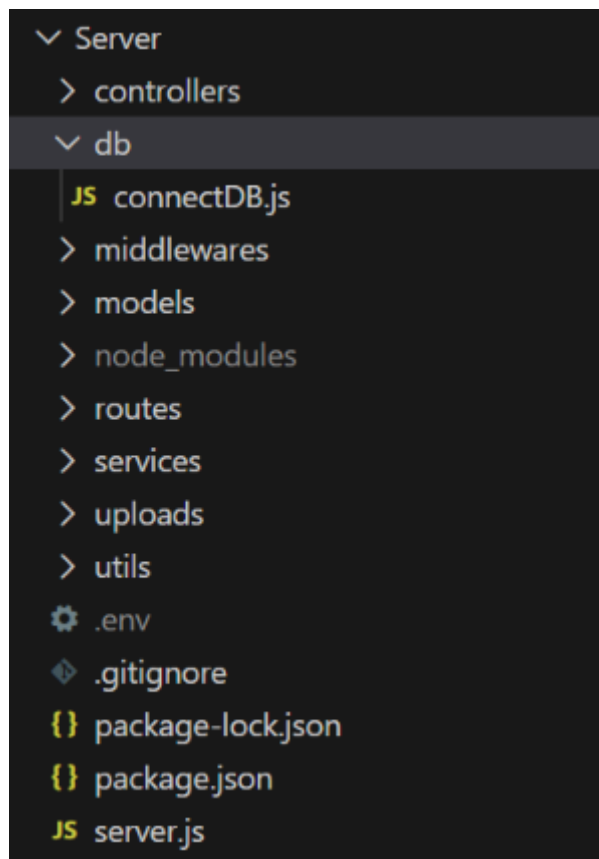
5.1 Client Structure(Frontend)



Description

Folder/File	Purpose
public	Static assets
api	Axios API configuration
assets	Images and icons
components	Reusable UI components
context	Global state management
pages	Application pages
App.jsx	Routing configuration
main.jsx	React entry point

5.2 Server Structure



Description

Folder/File	Purpose
controllers	Business logic
db	Database connection
middlewares	Authentication and validation
models	MongoDB schemas
routes	API endpoints
services	Shared business logic
uploads	Uploaded files
utils	Helper functions
server.js	Backend entry point

6. Running the Application

6.1 Starting the Frontend Server

Open a terminal and navigate to the client directory:

```
cd client
npm run dev
```

The frontend application will start on:

<http://localhost:5173>

6.2 Starting the Backend Server

Open another terminal and navigate to the server directory:

```
cd server
npm run dev
```

The backend server will start on:

<http://localhost:8000>

Successful startup message:

Server running on port 8000

MongoDB Connected Successfully

6.3 Accessing the Application

After both servers are running:

Component	URL
Frontend	http://localhost:5173
Backend API	http://localhost:8000

The user can now access the application through the frontend URL and interact with backend APIs.

7. API Documentation

7.1 User APIs

User Registration

Property	Value
Method	POST
Endpoint	/api/users/register
Description	Registers a new user

User Login

Property	Value
Method	POST
Endpoint	/api/users/login
Description	Authenticates user and returns JWT token

7.2 Cab APIs

Get Available Cabs

Property	Value
Method	GET
Endpoint	/api/cars
Description	Retrieves all available cabs

Add New Cab

Property	Value
Method	POST
Endpoint	/api/cars
Description	Adds a new cab to the system

Update Cab Details

Property	Value
Method	PUT
Endpoint	/api/cars/:id
Description	Updates existing cab information

Delete Cab

Property	Value
Method	DELETE
Endpoint	/api/cars/:id
Description	Deletes a cab from the system

7.3 Booking APIs

Create Booking

Property	Value
Method	POST
Endpoint	/api/bookings
Description	Creates a new booking

View User Bookings

Property	Value
Method	GET
Endpoint	/api/bookings/my
Description	Returns booking history of logged-in user

Cancel Booking

Property	Value
Method	DELETE
Endpoint	/api/bookings/:id
Description	Cancels an existing booking

7.4 Driver APIs

Get Driver Information

Property	Value
Method	GET
Endpoint	/api/drivers
Description	Retrieves driver information

Add Driver

Property	Value
Method	POST
Endpoint	/api/drivers
Description	Adds a new driver

7.5 Payment APIs

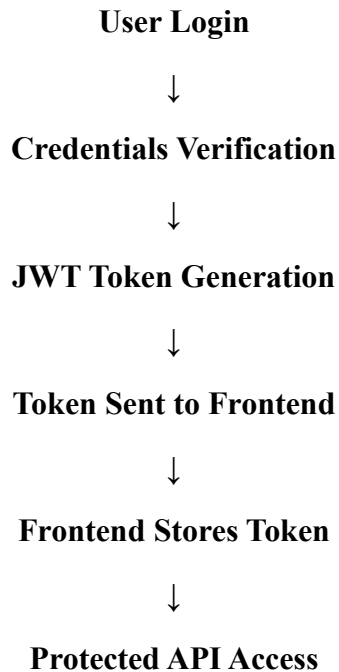
Process Payment

Property	Value
Method	POST
Endpoint	/api/payments
Description	Processes ride payments

8. Authentication

Authentication and authorization are implemented using **JWT (JSON Web Tokens)**.

Authentication Flow



Authorization

The application uses **Role-Based Access Control (RBAC)**.

Roles

- User
- Driver
- Admin

Each role has access only to the features relevant to their responsibilities.

Security Features

- Password hashing using bcrypt.
- JWT token validation using middleware.
- Protected routes for authenticated users.
- Role-based route access restrictions.

9. User Interface

The application provides separate interfaces for users, drivers, and administrators.

User Interface Screens

- Home Page
- Login Page
- Registration Page
- Cab Listing Page
- Booking Page
- Booking History Page

Driver Interface Screens

- Driver Dashboard
- Assigned Trips
- Trip History
- Availability Management

Admin Interface Screens

- Admin Dashboard
- Cab Management
- Driver Management
- Booking Management
- User Management

10. Testing

The Ucab application was tested to ensure that all modules work correctly and provide a smooth user experience.

Testing Strategy

The following testing approaches were used:

Functional Testing

Verified that all features perform according to requirements.

Examples:

- User registration
- User login
- Cab booking
- Booking cancellation
- Driver assignment

API Testing

Backend APIs were tested using **Postman**.

Tested APIs include:

- User APIs
- Cab APIs
- Booking APIs
- Driver APIs
- Payment APIs

Authentication Testing

Verified:

- JWT token generation
- Protected routes
- Role-based access
- Unauthorized access prevention

Database Testing

Verified:

- Data insertion

- Data retrieval
- Data updates
- Data deletion

User Interface Testing

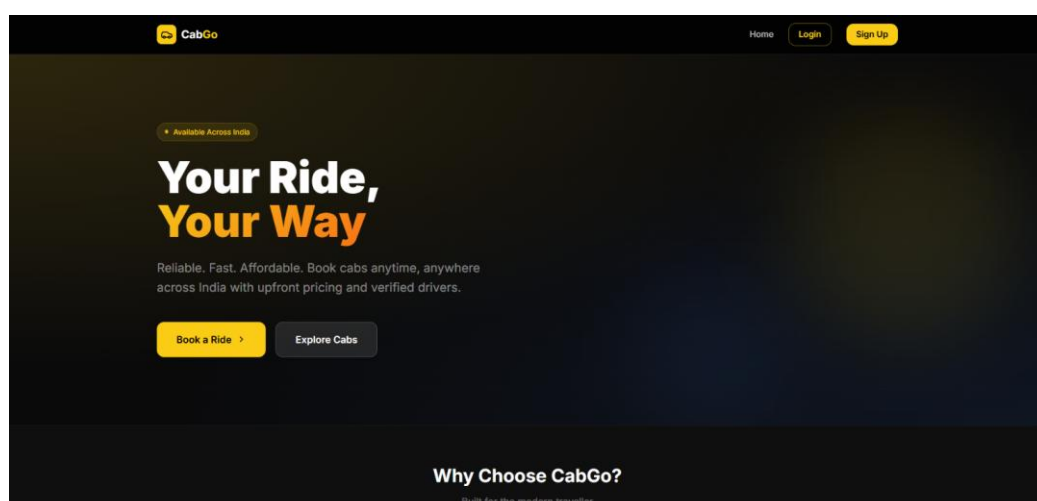
Verified:

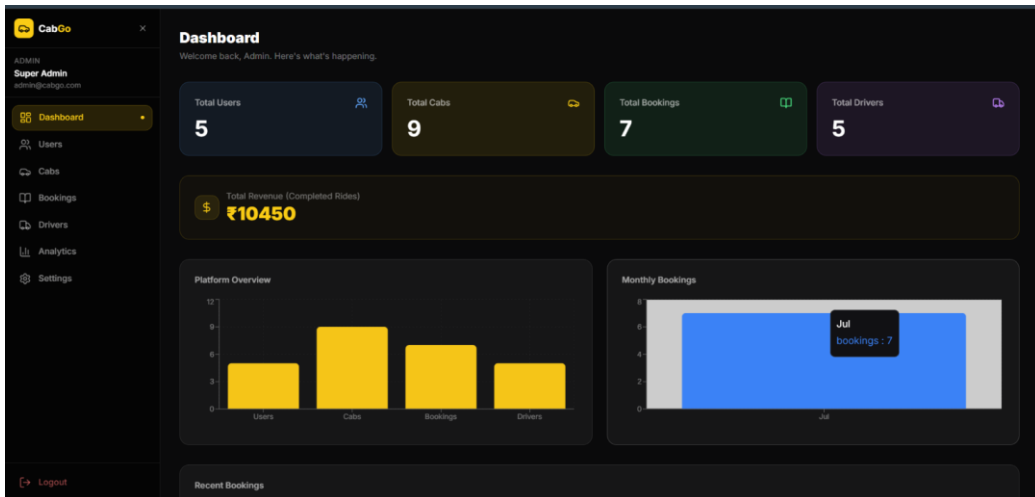
- Responsive design
- Navigation
- Form validations
- Error messages

Testing Tools Used

Tool	Purpose
Postman	API Testing
MongoDB Atlas	Database Verification
Browser Developer Tools	Frontend Debugging
Console Logs	Backend Debugging

11. Screenshots or Demo





CabGo

Welcome back

Sign in to your account

Email address

Password

→ Sign In

Don't have an account? [Register](#)

[Admin Portal](#) [Driver Portal](#)

CabGo

Home Book Cab My Bookings Scheduled **Mohini**

Good evening 🌙

Mohini

Where would you like to go today?

Book a Cab

My Bookings

Track Ride

Scheduled

Recent Bookings

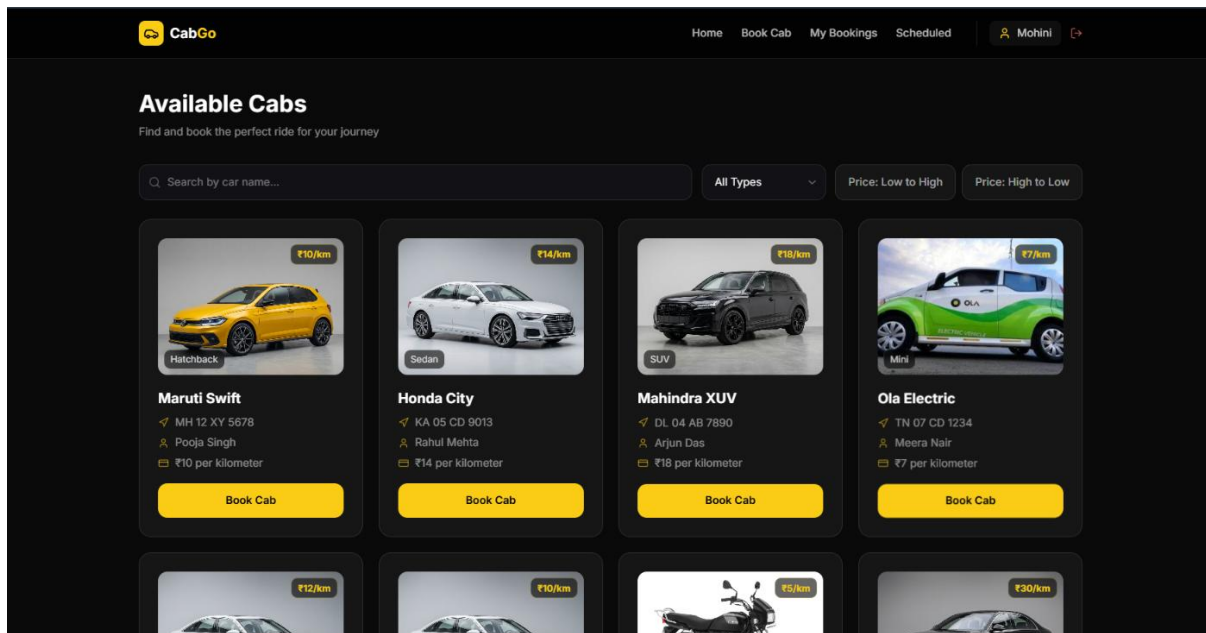
[View all >](#)

Vijayawada → Visakhapatnam
Honda City - 5/7/2026
₹4275
Pending

Visakhapatnam → Tirupati
Mahindra XUV - 5/7/2026
₹11200
Pending

CabGo © 2026 CabGo. All rights reserved. [Privacy Policy](#) [Terms of Service](#) [Support](#)

News for you



Demo For Frontend:

https://drive.google.com/file/d/14L2nCb4CUXJNuzgmYIRCctG8VN4sAS7T/view?usp=drive_link

Demo for Backend:

https://drive.google.com/file/d/15Vkm7zxgkE4idJ5cJuUUV7Q9smcWUPLX/view?usp=drive_link

Demo For Application Working:

https://drive.google.com/file/d/1Sp1b45Y-uWM8WR3thaVn7kDAqIj-QUFZ/view?usp=drive_link

GitHub Repository:

<https://github.com/mohinimutyala/cab-booking-mern>

12. Known Issues

The current version of the application has the following limitations:

- Payment gateway integration is still under development.
- Real-time GPS tracking is not available.
- Push notifications are not implemented.
- Email verification functionality is not available.

These limitations will be addressed in future versions of the application.

13. Future Enhancements

Several features can be added in future releases to improve the functionality and user experience of CarGo

Planned Enhancements

- Integration of online payment gateways.
- Real-time GPS tracking for rides.
- Mobile application development.
- Push notifications for ride updates.
- Email and SMS notifications.
- AI-based cab recommendation system.
- Dynamic pricing based on demand and traffic.
- In-app chat between drivers and users.
- Ride scheduling for future bookings.
- Rating and feedback system improvements.

Conclusion

Cargo is a full-stack cab booking application developed using the MERN stack that provides a seamless platform for users, drivers, and administrators.

The project successfully demonstrates:

- React frontend development
- REST API development using Express.js
- MongoDB database integration
- JWT-based authentication
- Role-based authorization
- Modular backend architecture

The application provides a strong foundation for building scalable transportation and ride-booking platforms while offering practical experience in modern web development technologies.

